ActiMM: Bridging Skip and Efficiency in GPU Sparse Convolution for 3D Point Cloud Processing

Anonymous Author(s)

Abstract

Sparse convolution is the dominant primitive for 3D point-cloud perception, yet existing GPU runtimes face a fundamental tradeoff between *computation skipping* and *execution efficiency*. Gather-GEMM-scatter approaches skip empty neighbors but suffer from per-offset launch overhead and load imbalance. Implicit GEMM approaches fuse all offsets into one efficient kernel but cannot skip empty neighbors, wasting compute proportional to sparsity.

We present ActiMM, a sparse convolution runtime built on *worklist-driven execution*. ActiMM precomputes all active neighbor pairs, groups them by kernel offset, and flattens the result into a contiguous tile stream executed by a single persistent-thread kernel. Empty offsets are skipped entirely; the stride loop provides automatic global load balance; and each tile is a pure dense GEMM. To recover data locality without heavy scheduling, ActiMM applies offset-major contiguous tile assignment and autotuned weight-stationary micro-kernels.

We evaluate ActiMM end-to-end on 3D object detection and semantic segmentation models across real-world point-cloud datasets. ActiMM achieves **TODO**× end-to-end speedup over TorchSparse++ on **TODO** and **TODO**× over SpConv v2 on **TODO**, while producing bit-identical results under FP32.

1 Introduction

Sparse convolution is the computational backbone of 3D point-cloud perception. LiDAR-based object detection [5, 6], semantic segmentation [1], and scene reconstruction all build on stacks of sparse convolution layers to process inherently sparse voxelized representations of 3D sensor data. In production models such as CenterPoint [6] and SECOND [5], sparse convolution accounts for **TODO**% of end-to-end inference time, making its runtime efficiency a first-order concern. These models operate at extreme sparsity: across a typical detection network on KITTI, the average per-layer active ratio is below **TODO**%, and even the densest layers rarely exceed **TODO**%.

A sparse convolution with kernel size k (typically 3; we use symmetric k^3 kernels throughout) examines k^3 neighbor offsets around each active output voxel. At each offset, the runtime checks whether the corresponding input neighbor is also active; if so, the input features at that neighbor are multiplied against the offset's weight slice and accumulated into the output. The set of active neighbors varies per voxel and per offset, producing an irregular, data-dependent computation pattern. The central runtime challenge is how to reorganize this irregular work into the regular, coalesced execution that GPUs require for high utilization.

Existing systems resolve this challenge by committing to one side of a fundamental tradeoff between *computation skipping* and *execution efficiency*.

The *implicit GEMM* approach (SpConv v2 [3], TorchSparse++ [4]) treats every active output voxel as if all k^3 neighbors were present: it concatenates the full neighborhood into one large matrix and performs a single fused dense matrix multiply across all offsets simultaneously. This yields high GPU utilization and Tensor Core compatibility, but it cannot skip empty neighbors; missing neighbors simply contribute zero-valued rows that still consume compute cycles and memory bandwidth. Profiling a CenterPoint backbone on KITTI reveals that **TODO**% of the fused multiply-accumulate operations involve such zero-valued inputs, wasted compute that scales with the kernel volume rather than the actual connectivity.

The *per-offset gather-GEMM-scatter* approach (MinkowskiEngine [2]) takes the opposite route: it loops over the k^3 offsets one by one, collects only the active pairs at each offset, and runs a separate matrix multiply for each. This avoids wasted computation on empty neighbors. However, it pays a steep price: up to k^3 separate kernel launches per convolution layer, severe load imbalance across offsets, and either a naive in-kernel matrix multiply or an extra copy into contiguous buffers before handing off to a tuned dense GEMM library. On the same CenterPoint backbone, gather-GEMM-scatter spends **TODO**% of its wall time on launch overhead and data movement rather than useful arithmetic.

At the low densities that dominate real workloads, neither side works well. Implicit GEMM wastes overwhelmingly on empty neighbors. Gather-GEMM-scatter skips the empties, but each per-offset matrix multiply is too small to saturate the GPU.

We observe that both sides of this tradeoff organize execution around *offsets*: implicit GEMM iterates all offsets per output point, while gather-GEMM-scatter launches one kernel per offset. Neither organizes execution around the *actual active work*.

ActiMM takes a different approach: *worklist-driven execution*. We precompute all active `(input, output)` neighbor pairs across every offset, group them by offset, and flatten the groups into a contiguous stream of dense GEMM tiles (a worklist). A single persistent-thread kernel then stride-loops over this worklist. This design has three key properties:

- **Exact work.** Offsets with no active pairs produce zero tiles and are skipped entirely; computation is strictly proportional to active neighbor pairs.
- **Global load balance.** All offsets share one kernel launch; the stride loop spreads tiles evenly across thread blocks regardless of how unevenly pairs distribute across offsets.
- **Dense inner compute.** Each tile is a standard dense matrix multiply, fully GPU-friendly and Tensor Core compatible.

Exact-work execution is not free. Once convolution is decomposed into per-offset active pairs, the runtime becomes more fetch-on-demand: input features are gathered indirectly via pair indices, output accumulation spans multiple offsets (requiring atomic additions), and tile ordering strongly affects cache reuse. The challenge is how to recover enough data locality *without* sacrificing exact work or breaking global load balance.

ActiMM addresses this with three low-complexity dataflow choices, rather than a heavy scheduling framework:

- **Offset-major contiguous tile assignment.** Tiles in the worklist are ordered so that consecutive iterations of the stride loop tend to share the same kernel offset. This improves L2 cache locality for input features while preserving the load-balance property of the global stride loop.
- **Weight-fragment persistence.** When consecutive tiles share the same offset, the weight slice can be kept live in registers instead of being reloaded. This turns the offset-major ordering into a direct reduction in memory traffic.
- We use **two-level sparsity-aware autotuning** to find the optimal tile shape and micro-kernel variant with zero overhead on the critical path as runtime sparsity level varies. Offline sweeps build both a per-layer config table indexed by sparsity bucket and an input-to-layer sparsity mapping; at inference time, the runtime reads the input active ratio and selects the best config for every layer on the host alone, without any per-layer profiling or device-to-host synchronization.

Table 1 previews the operator-level impact of this execution model on a controlled microbenchmark. On an RTX 2080 Ti with FP32 precision, ActiMM outperforms all existing sparse convolution systems across the entire active-ratio range tested. At 1% active ratio (typical for LiDAR detection layers), ActiMM is 5.5× faster than the fastest existing system. The advantage holds up to 30%, where ActiMM still maintains a 1.8× margin. End-to-end model-level evaluation on detection and segmentation benchmarks is presented in §??.

This paper makes the following contributions:

Table 1. Operator-level kernel latency (ms) for SubM 3×3×3 convolution, C_{in}=C_{out}=64, spatial 128×128×8, FP32, RTX 2080 Ti. Speedup is over the fastest existing system at each ratio. End-to-end model evaluation with real datasets is in §?? (*in progress*).

Active ratio	ActiMM	SpConv v2	TorchSparse++	Minkowski-Engine	Speedup
0.1%	**0.012**	0.421	0.240	0.230	19.2×
0.5%	**0.045**	0.374	0.240	0.323	5.3×
1%	**0.045**	0.403	0.246	0.379	5.5×
5%	**0.026**	0.436	0.242	0.386	9.3×
10%	**0.067**	0.551	0.377	0.445	5.6×
20%	**0.240**	1.251	0.685	0.908	2.9×
30%	**0.531**	1.544	0.945	1.703	1.8×

1. A **worklist-driven execution model** for sparse convolution that achieves computation proportional to active neighbor pairs within a single persistent-thread kernel, unifying skip and efficiency without the trade-offs of implicit GEMM or gather-GEMM-scatter.
2. **Lightweight locality recovery** through offset-major tile assignment and autotuned weight-stationary micro-kernels, recovering data reuse without heavy scheduling.
3. An end-to-end evaluation on 3D object detection and semantic segmentation models showing **TODO**× speedup over the fastest existing sparse convolution runtime, with numerically identical outputs.

References

[1] Jens Behley, Martin Garbade, Andres Milioto, Jan Quenzel, Sven Behnke, Cyrill Stachniss, and Jurgen Gall. 2019. SemanticKITTI: A Dataset for Semantic Scene Understanding of LiDAR Sequences. In *ICCV*.

[2] Christopher Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *CVPR*.

[3] Lue Fan, Feng Wang, Ziqi Pang, Lei Zhu, Zhicheng Su, Fang Chen, Naiyan Wang, and Zhaoxiang Zhang. 2022. Embracing Single Stride 3D Object Detector with Sparse Transformer. In *CVPR*.

[4] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *MICRO*.

[5] Yan Yan, Yuxing Mao, and Bo Li. 2018. SECOND: Sparsely Embedded Convolutional Detection. *Sensors*.

[6] Tianwei Yin, Xingyi Zhou, and Philipp Krähenbühl. 2021. Center-based 3D Object Detection and Tracking. In *CVPR*.